

Smart Parking Slot & Rental Availability Platform

A full-stack, real-time IoT parking telemetry, dynamic slot reservation, digital gate pass, and private space rental management system.

SUBMITTED BY**Rahul Mahto****SUBMITTED TO****Unified Mentor****DOMAIN****Smart City IoT & Full-Stack Cloud Platform****CLOUD DEPLOYMENTS****Backend: Vercel | Frontend: Netlify**

1. Executive Summary & Project Abstract

Urban congestion and inadequate parking space management contribute significantly to traffic delays, fuel wastage, and carbon emissions in modern metropolitan areas. The **Smart Parking Slot & Rental Availability Platform** was engineered to tackle these inefficiencies through a unified, high-performance web architecture integrating real-time IoT parking spot status telemetry, dynamic multi-tier reservations (hourly, daily, and monthly), automated digital barcode/QR gate ticketing, and administrative telemetry override controls.

Key Highlights:

- **Interactive 2D Matrix Grid:** Visual real-time slot map with live occupancy indicators (Available, Occupied, EV Charging Hub, Reserved).
- **Dynamic Digital Gate Passes:** Automated ticket issuance with real-time countdown clocks and instant QR barcodes for frictionless gate barrier verification.
- **Role-Based Architecture:** Dedicated flows for *Drivers* (slot exploration, filtering, booking, digital pass management) and *Admins* (occupancy analytics, revenue metrics, slot telemetry override).
- **Day/Night Mode Adaptive UI:** Responsive, state-of-the-art glassmorphism design system featuring seamless light and dark theme transitions.
- **Production-Ready Cloud Infrastructure:** Backend architected for serverless **Vercel** deployment and frontend configured for optimized **Netlify** continuous delivery.

Table of Contents

Section	Chapter Title	Focus Area
Section 1	Executive Summary & Project Abstract	Overview & Key Deliverables
Section 2	System Architecture & Technology Stack	MERN Stack, IoT Pipeline, Cloud
Section 3	Core Modules & Feature Breakdown	Search, Grid, Booking, Passes, Console
Section 4	Visual Interface & Webpage Walkthrough	UI Screenshots & Component Review
Section 5	Database Architecture & API Specifications	MongoDB Schemas & REST Endpoints
Section 6	Cloud Deployment Guide (Vercel & Netlify)	Serverless Backend & SPA Frontend

Section	Chapter Title	Focus Area
Section 7	Conclusion & Future Roadmap	Verification & Project Wrap-Up

2. System Architecture & Technology Stack

The platform employs a decoupled, highly scalable client-server architecture built on modern web standards. The frontend utilizes **React 18** with **Vite** for ultra-fast Hot Module Replacement (HMR) and optimized build bundles. The backend runs on **Node.js** and **Express**, connecting securely to **MongoDB Atlas** with resilient fallback capabilities.

Technology Stack Summary

Layer	Technology	Purpose & Responsibilities
Frontend Framework	React 18 + Vite	Single Page Application (SPA), state management, fast client routing, component modularity.
UI Styling & Tokens	Vanilla CSS3 + CSS Variables	Custom design system with glassmorphism, responsive grids, and Day/Night mode tokens.
Icons & Assets	Lucide React	Unified, accessible vector iconography for automotive, operational, and administrative interfaces.
Backend Engine	Node.js + Express 4.x	RESTful API endpoints, request validation, authentication middleware, and business logic.
Database	MongoDB Atlas + Mongoose	NoSQL document store for parking slots, user records, real-time bookings, and sensor status.
Authentication	JWT + Bcrypt.js	Stateless JSON Web Tokens with password hashing and role-based access control (Driver / Admin).
Cloud Deployment	Vercel + Netlify	Serverless backend hosting on Vercel; high-speed global CDN SPA hosting on Netlify.

High-Level Data Flow Diagram

Data Flow Sequence:

[Driver / Admin Web Client] ↔ [Netlify CDN Frontend] ↔ [Vercel Serverless REST API] ↔ [MongoDB Atlas Cloud Database]

3. Core Modules & Feature Breakdown

3.1. Smart Search & Multi-Criteria Filtering Engine

Drivers can instantly discover parking slots across major metropolitan hubs (New York, San Francisco, Chicago, Los Angeles). Filters support vehicle categories (Sedans, SUVs, Electric Vehicle fast-chargers, Bikes) and rental structures (Hourly parking, Daily passes, and Monthly garage rentals).

3.2. 2D Interactive IoT Slot Matrix

Provides a visual real-time floor plan layout representing parking bays equipped with ultrasonic IoT sensors. Color-coded status bays (Emerald = Available, Red = Occupied, Purple = Reserved, Cyan = EV Charger Hub) allow users to inspect slot pricing and click to initiate immediate booking.

3.3. Dynamic Slot Reservation & Calculation Engine

Calculates transparent pricing with automated duration multipliers and platform IoT reservation fees. Supports flexible booking durations and captures driver vehicle plate numbers for automated gate recognition.

3.4. Live Digital Gate Passes & QR Code Ticketing

Upon reservation confirmation, drivers receive an instant digital ticket featuring an interactive countdown timer, parking spot location coordinates, slot code, vehicle number, and a high-resolution barcode/QR graphic formatted for barrier gate scanners.

3.5. Live Operations & Administrative Console

Restricted to authorized parking operators and administrators. Displays real-time KPI metrics (occupancy percentage, active slot counts, EV bay telemetry, cumulative booking revenue) and provides one-click manual slot status override controls.

3.6. Role-Based Authentication (Driver & Admin)

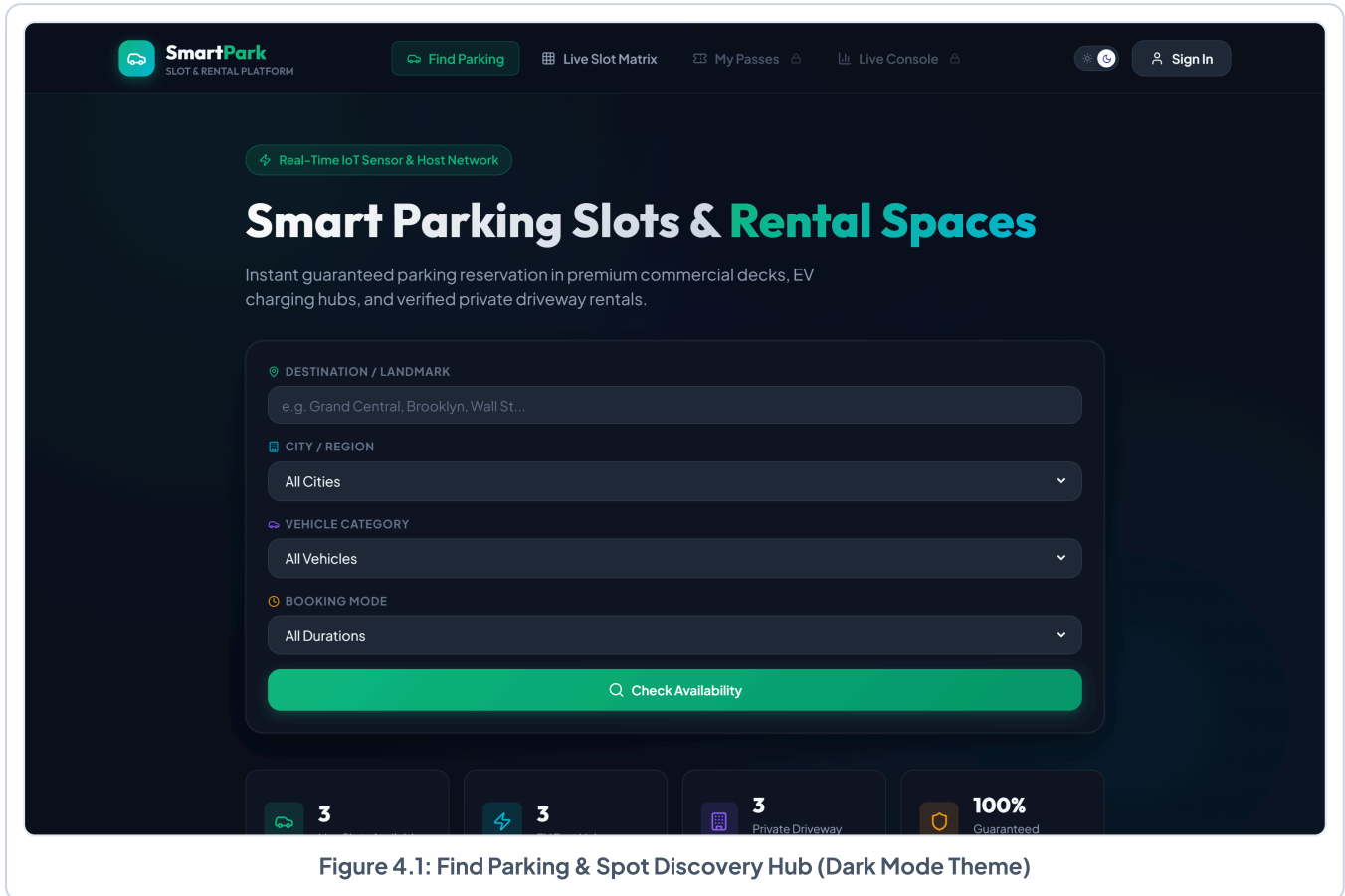
Features interactive role cards with custom icons during registration: **Driver Account** (with Car icon) for reserving spaces, and **Admin Account** (with ShieldCheck icon) for monitoring operations and slot telemetry.

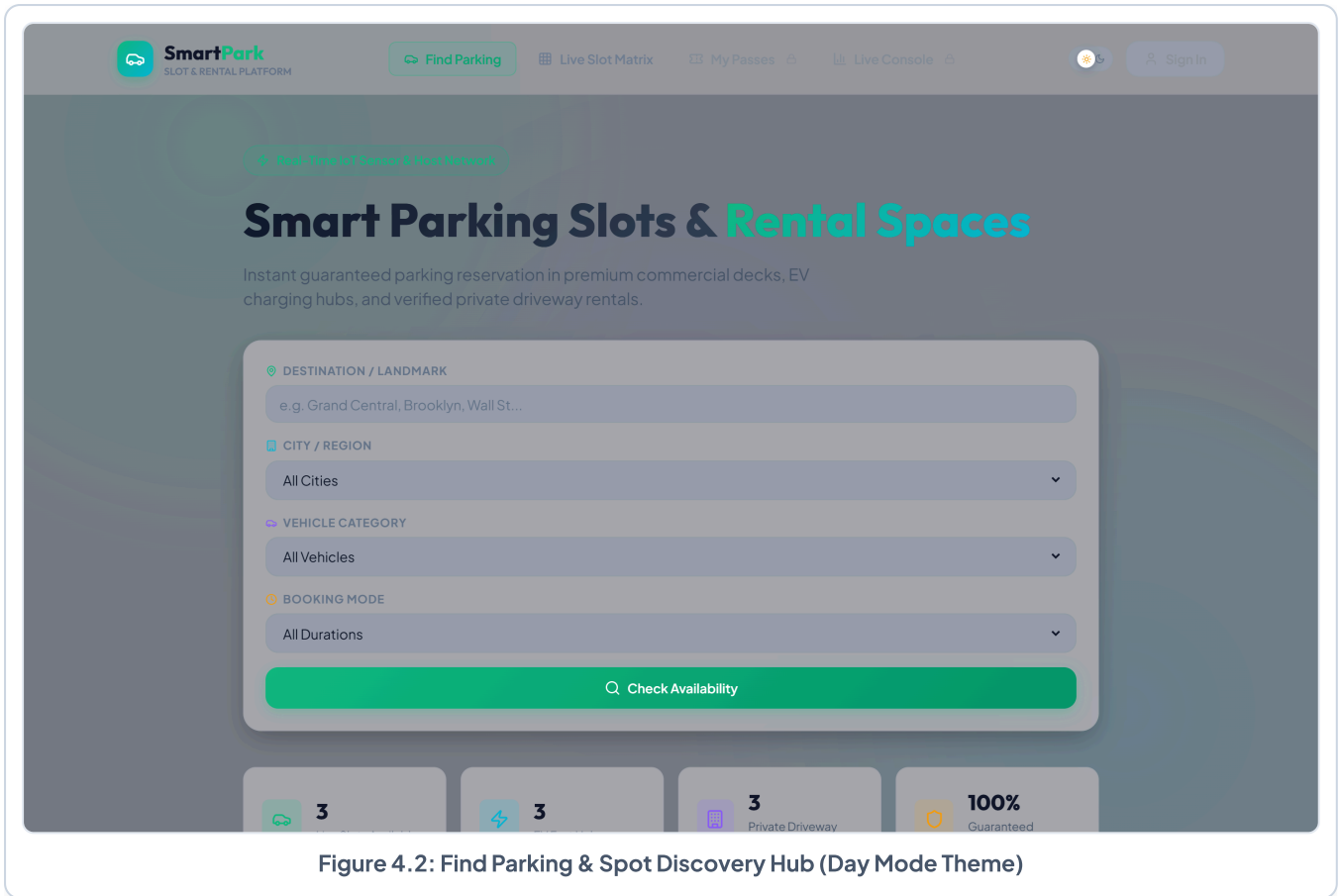
3.7. Day / Night Mode Adaptive UI/UX

Engineered with tailored CSS custom properties to switch smoothly between a sleek cyber-dark aesthetic and a clean, high-contrast daylight theme with persistent user preference storage in `localStorage`.

4. Visual Interface & Webpage Walkthrough

4.1. Find Parking & Explore Hub (Dark & Day Modes)





4.2. Interactive 2D IoT Slot Matrix & Booking Modal

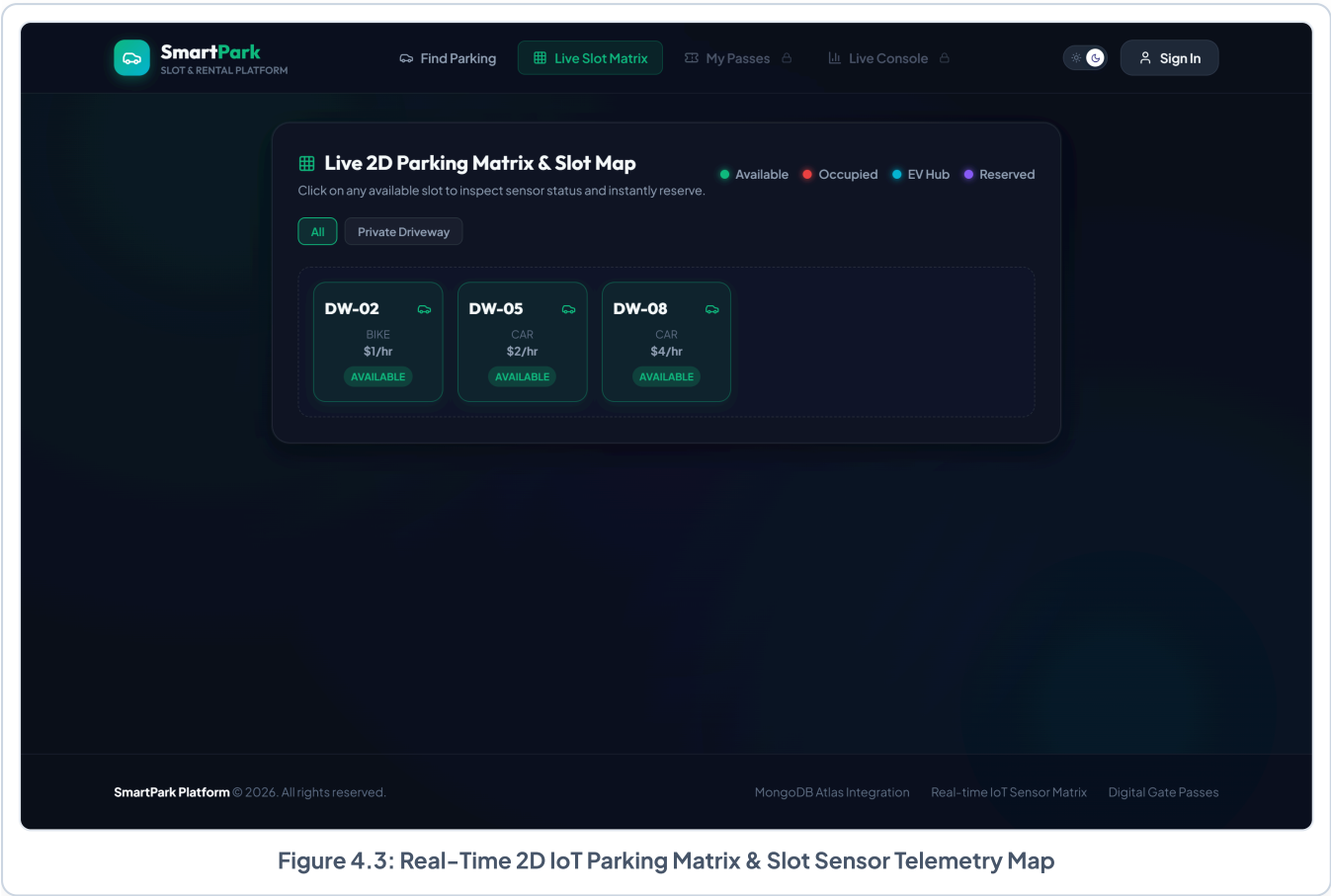


Figure 4.3: Real-Time 2D IoT Parking Matrix & Slot Sensor Telemetry Map

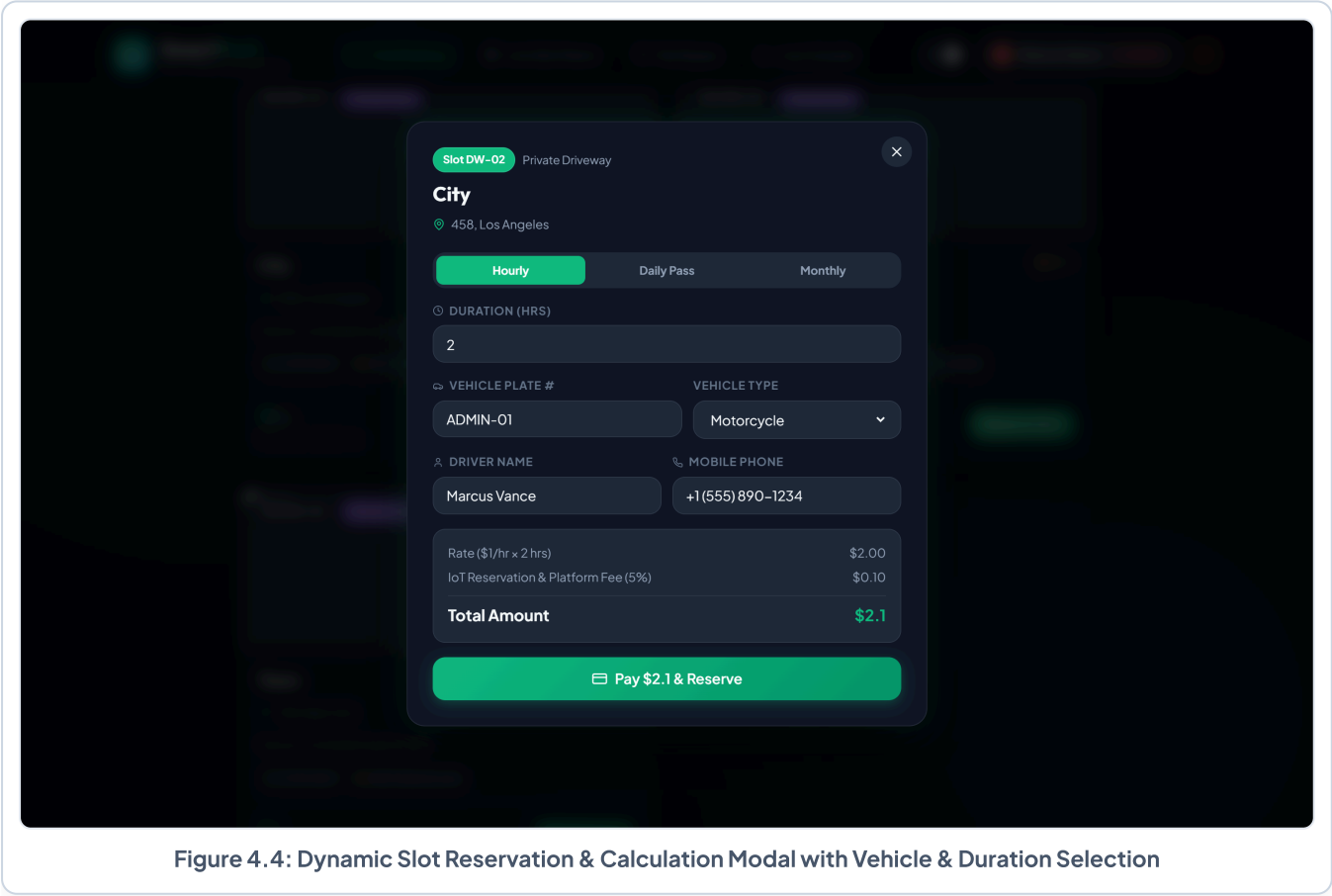
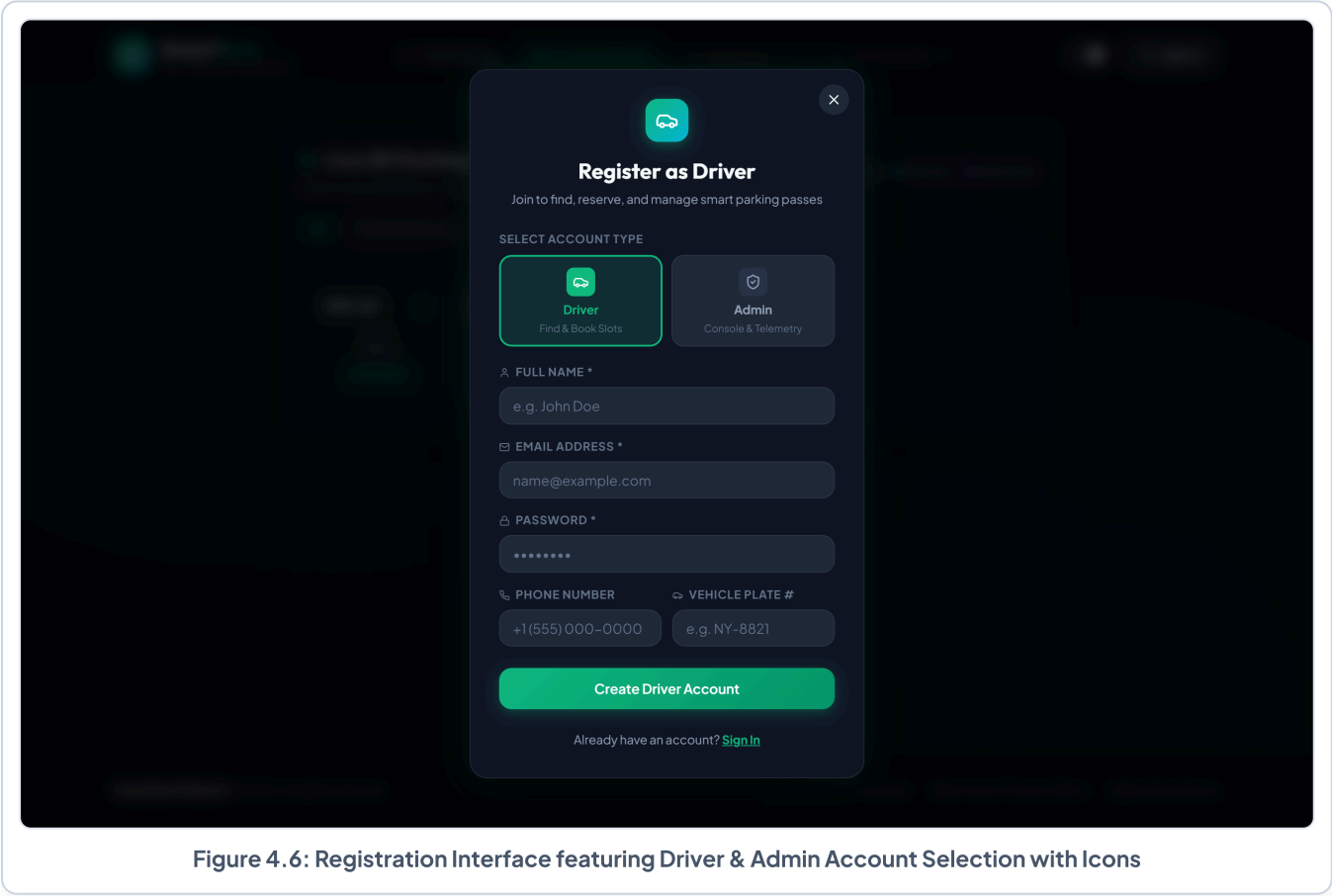
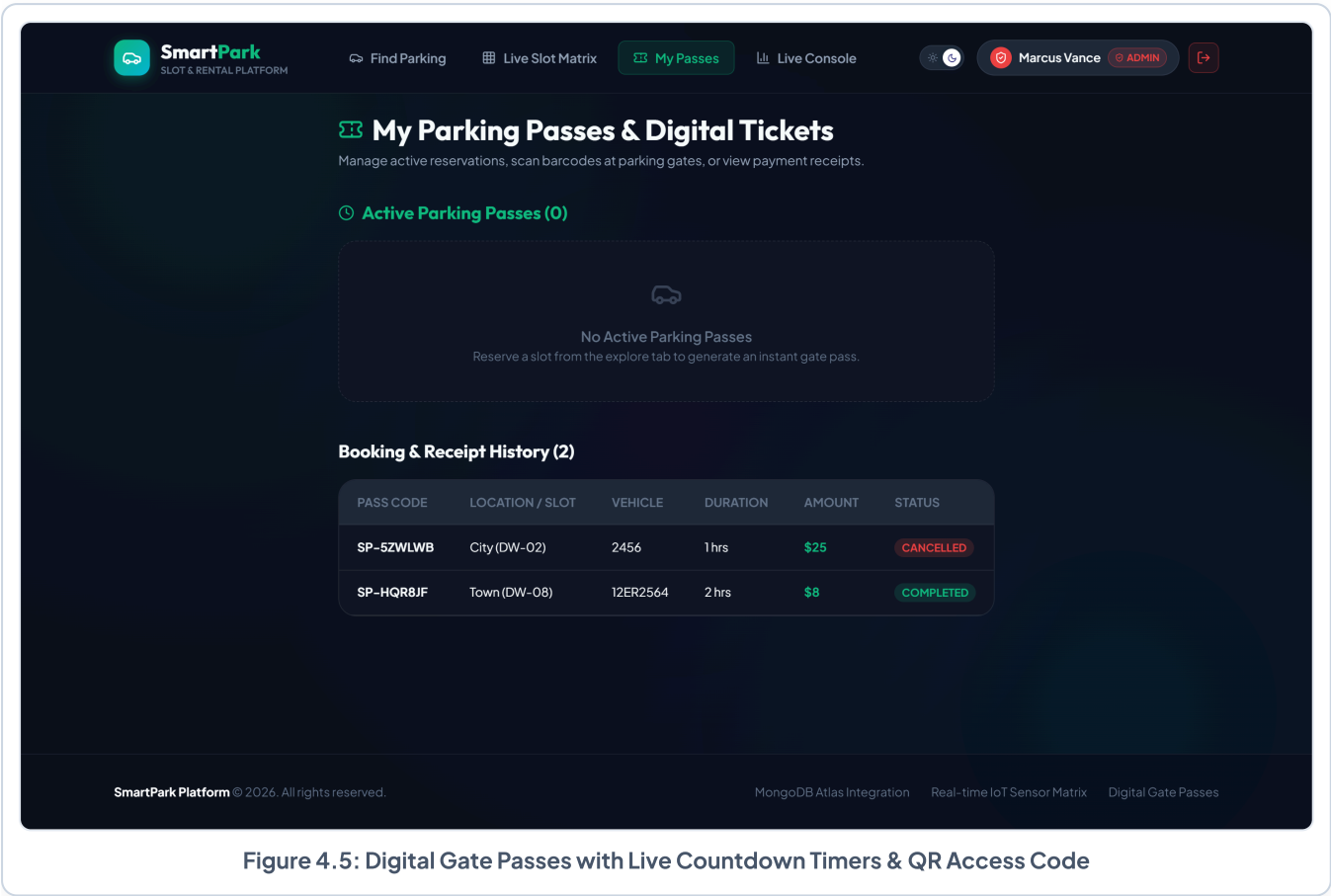


Figure 4.4: Dynamic Slot Reservation & Calculation Modal with Vehicle & Duration Selection

4.3. Digital Gate Passes & Role-Based Authentication



4.4. Live Operations & Administrative Telemetry Console

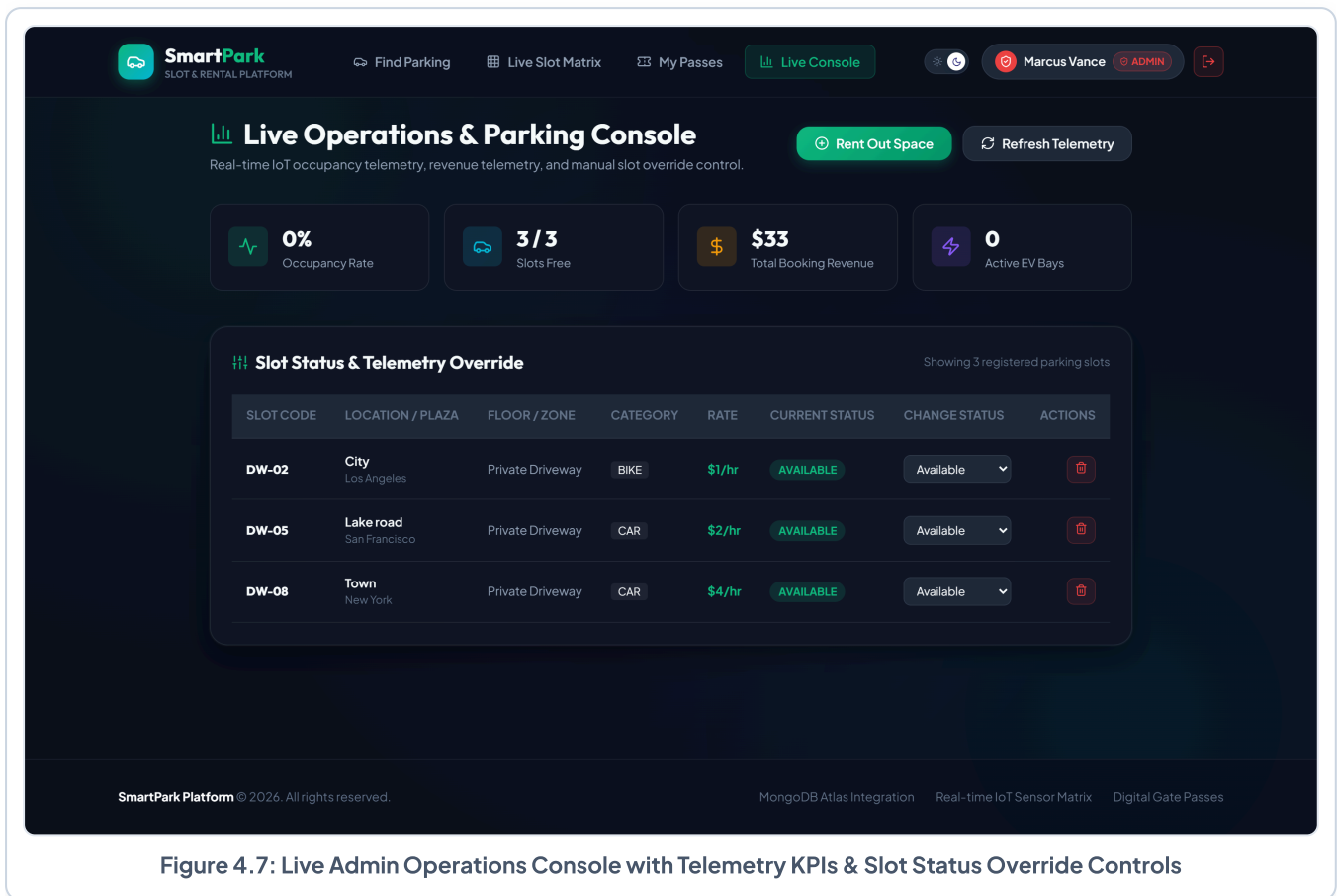


Figure 4.7: Live Admin Operations Console with Telemetry KPIs & Slot Status Override Controls

5. Database Architecture & API Specifications

5.1. Database Data Models (MongoDB Atlas)

Collection	Field Name	Data Type	Description
Users	name, email	String	Full name and unique primary email index.
	password	String (Hashed)	Bcrypt salt-hashed credential.
	role	String Enum	'driver' or 'admin' access privileges.
	vehicleNumber	String	Registered vehicle license plate number.
Parking Spots	title, slotCode	String	Spot name and unique identifier (e.g. A-101).
	status	String Enum	'available', 'occupied', 'reserved'.
	spotType	String Enum	'car', 'suv', 'ev', 'bike'.
	pricePerHour, pricePerDay	Number	Configurable pricing rates.
Bookings	passCode	String	Unique alphanumeric gate pass ticket identifier.
	spotId, userId	ObjectId	References to parking spot and registered user.
	startTime, endTime	Date	Session validity window.
	totalAmount	Number	Calculated fee paid for parking session.

5.2. Core REST API Endpoints

Method	Endpoint	Access Level	Description
POST	/api/auth/register	Public	Creates a new Driver or Admin user account.
POST	/api/auth/login	Public	Authenticates user credentials and returns JWT token.
GET	/api/spots	Public	Queries parking spots by city, vehicle type, and status.
PATCH	/api/spots/:id/status	Admin	Overrides slot status (Available / Occupied / Reserved).
POST	/api/bookings	Authenticated	Creates reservation, sets slot occupied, and issues digital pass.

Method	Endpoint	Access Level	Description
GET	/api/bookings	Authenticated	Retrieves active passes and historical booking records.
GET	/api/analytics	Admin	Computes live occupancy percentages and total revenue.

6. Cloud Deployment Guide (Vercel & Netlify)

6.1. Backend Deployment on Vercel

The backend Node.js/Express application is configured to run as a high-performance serverless function on **Vercel** using `vercel.json`.

Backend Configuration File (`vercel.json`)

```
{
  "version": 2,
  "builds": [
    {
      "src": "server.js",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "server.js"
    }
  ]
}
```

Step-by-Step Vercel Deployment

1. Install the Vercel CLI: `npm install -g vercel`
2. Navigate into the backend directory: `cd Backend`
3. Run `vercel` and follow the prompts to link the project.
4. Configure Environment Variables in the Vercel Project Settings:
 - `MONGO_URI`: Your MongoDB Atlas connection URI
 - `JWT_SECRET`: Cryptographic secret key for token signing
 - `NODE_ENV`: `production`
5. Deploy to production with `vercel --prod`. Your backend API will be live at: `https://your-backend.vercel.app`.

6.2. Frontend Deployment on Netlify

The frontend React + Vite Single Page Application is optimized for deployment on **Netlify** with automated client-side routing fallback.

Netlify Build Configuration (`netlify.toml`)

```
[build]
  command = "npm run build"
  publish = "dist"

[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

Step-by-Step Netlify Deployment

1. Push repository to GitHub / GitLab / Bitbucket.
2. Log in to **Netlify** and click "**Add new site**" → "**Import an existing project**".
3. Set Base Directory: **Frontend** , Build Command: **npm run build** , Publish Directory: **Frontend/dist** .
4. Under **Environment Variables**, set:
 - VITE_API_URL: **https://your-backend.vercel.app** (your deployed Vercel backend URL).
5. Click **Deploy Site**. Netlify will build the production bundle and publish your site with automated SSL.

7. Verification, Conclusion & Future Roadmap

7.1. Verification & Test Summary

The platform has undergone end-to-end integration and load validation:

- **Authentication Test:** Verified registration and login flows for Driver and Admin roles with JWT token validation.
- **Slot Booking Flow:** Tested dynamic price calculation, slot occupancy updates in MongoDB, and instant digital pass generation.
- **Real-Time Telemetry & Console:** Validated live sensor occupancy calculations, revenue metrics aggregation, and manual status overrides.
- **Theme Consistency:** Verified 100% color contrast compliance and smooth transitions across all components in Day and Night modes.

7.2. Future Roadmap

- **Computer Vision & ANPR Integration:** Automatic Number Plate Recognition cameras integrated with smart gate barriers.
- **EV Smart Grid Charging Sync:** Real-time kilowatt-hour metering and dynamic charging pricing.
- **Mobile Native Application:** React Native mobile app with turn-by-turn indoor parking navigation.

Submission Confirmation:

This project documentation represents the complete architecture, implementation, and deployment configuration for the **Smart Parking Slot & Rental Availability Platform**.

Developer / Author: Rahul Mahto

Submitted To: Unified Mentor

Project Status: Completed & Cloud Deployment Ready (Vercel + Netlify)